

Çağla Ökmen

25435004059

İçerik

Fİnal Proje Dökümantasyonu:	1
AjanRL.cs	1
QBombENV_sc.cs	12
Pathfinder.cs	18
SimpleMLP.cs	26
ExperienceReplay.cs	29
SimpleBomb.cs	30
CurriculumTarget.cs	32
ExperimentManager.cs	37
TrainingAnalyticsUI.cs	40

Fİnal Proje Dökümantasyonu:

GitHub linki: <https://github.com/CaglaOkmen/QBomb>

AjanRL.cs

Oyunun ana kontrol merkezidir; çevreden aldığı verileri (duvarlar, bombalar, tehlike haritası) sınır ağına girdi olarak sunar, Q-öğrenme algoritmasıyla ajanın hangi aksiyonu (hareket, bomba bırakma, bekleme) yapacağına karar verir ve eğitim sürecini yönetir.

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using System.Linq;
using System.IO;

public class AjanRL : MonoBehaviour
{
    [Header("Hiperparametreler")]
    public int maxEpisode = 4000;
    public float learningRate = 0.0001f;
    public float gamma = 0.99f;
    public float epsilon = 1.0f;
    public float epsilonDecay = 0.9991f;
    public float minEpsilon = 0.05f;

    [Header("MLP")]
    public int hiddenLayerSize = 64;
    public int batchSize = 32;

    [Header("Train modu")]
    public bool isTraining = false;

    public string currentSaveFileName = "default_model.json";

    [Header("Güvenlik katmanı")]
    public bool useSafetyRule = true;

    [Header("Gorselleme")]
    public float currentEpisodeReward = 0;
    public int currentEpisodeSteps = 0;
    public int currentEpisodeIndex = 0;

    // --- CSV raporlama için
    [Header("Analiz Verileri (Last Episode)")]
    public string lastOutcome = "None";
```

```

public string lastDeathType = "None";
public float lastExplorationRate = 0f;
public int lastBombsDropped = 0;
public int lastSafetyDangerSteps = 0;

// Epsilon ve Faz Yonetimi
private int lastPhaseTracker = -1;
private CurriculumTarget targetScript;

// 13 Hucre
private readonly Vector2Int[] observationPattern = new Vector2Int[]
{
    // Mesafe 0 (Merkez)
    new Vector2Int(0, 0),

    // Mesafe 1 (3x3 Karenin geri kalani - 8 hucre)
    new Vector2Int(0, 1), // Yukari
    new Vector2Int(0, -1), // Asagi
    new Vector2Int(1, 0), // Sag
    new Vector2Int(-1, 0), // Sol
    new Vector2Int(1, 1), // Sag-Ust
    new Vector2Int(-1, 1), // Sol-Ust
    new Vector2Int(1, -1), // Sag-Alt
    new Vector2Int(-1, -1), // Sol-Alt

    // Mesafe 2 (Sadece Arti Sekli - 4 hucre)
    new Vector2Int(0, 2), // Yukari 2
    new Vector2Int(0, -2), // Asagi 2
    new Vector2Int(2, 0), // Sag 2
    new Vector2Int(-2, 0) // Sol 2
};

private int inputSize;

private SimpleMLP mlp;
private ReplayBuffer replayBuffer;
private QBombENV_sc env;
private Pathfinder pathfinder;

// İstatistikler
private int winCount = 0;
private int deathCount = 0;
private int timeoutCount = 0;

private int[] actionCounts;

public void SetExperimentConfig(string fileName, bool safetyEnabled)
{
    currentSaveFileName = fileName;
    useSafetyRule = safetyEnabled;

    // kayitli modeli yukle, yoksa sifir ag baslat
    LoadNetwork();
}

void Awake()
{
    // 6 kanal (IsWall, IsBreakable, IsTarget, IsBomb, BombTimer,
    IsInDanger)
    int gridCells = observationPattern.Length;
    int channelsPerCell = 6;
    int gridInputs = gridCells * channelsPerCell;

```

```

        // Global inputs: Target X,Y, Targete uzaklık, tehlikede mi, Ajan
        ve Target bombalai aktif mi, Yol acik kapali
        int globalInputs = 5;

        inputSize = gridInputs + globalInputs;

        env = FindObjectOfType<QBombENV_sc>();
        pathfinder = gameObject.AddComponent<Pathfinder>();
        targetScript = FindObjectOfType<CurriculumTarget>();
    }

    void Start()
    {
        Time.timeScale = 25f;

        actionCounts = new int[env.numActions];
        mlp = new SimpleMLP(inputSize, hiddenLayerSize, env.numActions,
learningRate);
        replayBuffer = new ReplayBuffer(10000);

        if (!isTraining) LoadNetwork();
    }

    void Update()
    {
        if (Input.GetKeyDown(KeyCode.Space)) { StopAllCoroutines();
StartCoroutine(AjanTraining()); }
        if (Input.GetKeyDown(KeyCode.K)) SaveNetwork();
    }

    float[] GetObservation()
    {
        float[] observations = new float[inputSize];
        int index = 0;
        float cellSize = env.cellSize;
        Vector2 agentPos = transform.position;

        // 1 birim tam cevre ve 2 birim uzaklik + seklindeki alanlar
        foreach (Vector2Int offset in observationPattern)
        {
            Vector2 scanPos = agentPos + new Vector2(offset.x * cellSize,
offset.y * cellSize);

            bool isWall = false;
            bool isBreakable = false;
            bool isTarget = false;
            bool isBomb = false;
            float bombTimer = 0f; // 0: Yok, 0-1 arasi: patlama suresi
            bool isDangerous = false;

            Collider2D hit = Physics2D.OverlapBox(scanPos, Vector2.one *
(cellSize * 0.9f), 0);
            if (hit != null)
            {
                if (hit.CompareTag("wall")) isWall = true;
                else if (hit.CompareTag("breakable")) isBreakable = true;
                else if (hit.CompareTag("target")) isTarget = true;
                else if (hit.CompareTag("bomb"))
                {
                    isBomb = true;
                    // Bombanın patlamasına ne kadar kaldı
                    SimpleBomb sb = hit.GetComponent<SimpleBomb>();

```

```

        // +1 ekleyerek 0 olmamasini sagliyoruz (0 bomba yok
demek)
        // Ornegin 3 adimli bombada: 1/4, 2/4, 3/4, 4/4 gibi
degerler alır
        bombTimer = (float)(sb.currentStep + 1) /
(sb.explosionSteps + 1);
    }

    int checkX = Mathf.RoundToInt(scanPos.x / cellSize);
    int checkY = Mathf.RoundToInt(scanPos.y / cellSize);

    if (checkX >= 0 && checkX < env.width && checkY >= 0 && checkY
< env.height)
    {
        isDangerous = env.dangerMap[checkX, checkY];
    }

    // Onehot kodlama ve Bomba zamani
    observations[index++] = isWall ? 1.0f : 0.0f;
    observations[index++] = isBreakable ? 1.0f : 0.0f;
    observations[index++] = isTarget ? 1.0f : 0.0f;
    observations[index++] = isBomb ? 1.0f : 0.0f;
    observations[index++] = bombTimer;
    observations[index++] = isDangerous ? 1.0f : 0.0f;

}

// hedefin yonu
(float)env.width;
    float relativeTargetX = (env.targetX - env.gridX) /
(float)env.height;
    float relativeTargetY = (env.targetY - env.gridY) /
(float)env.height;
    observations[index++] = relativeTargetX;
    observations[index++] = relativeTargetY;

// hedefin mesafesi
    float manhattanDist = (Mathf.Abs(env.targetX - env.gridX) +
Mathf.Abs(env.targetY - env.gridY));
    float normalizedDist = manhattanDist / (env.width + env.height);
    observations[index++] = normalizedDist;

// Tehlike durumu
    bool inDanger = pathfinder.IsInDanger(env.gridX, env.gridY);
    observations[index++] = inDanger ? 1.0f : 0.0f;

// Yol Acik, kapali
    observations[index++] = (pathfinder.currentPathType ==
Pathfinder.PathType.Clear) ? 1.0f : 0.0f;

    return observations;
}

IEnumerator AjanTraining()
{
    int maxStepsPerEpisode = 600;

    winCount = 0; deathCount = 0; timeoutCount = 0;

    epsilon = 1.0f;
    lastPhaseTracker = -1; // Faz takibini sifirla

    print($"=== TRAINING START ({currentSaveFileName}) ===");

```

```

isTraining = true;

// Egitim baslarken UI sayacini temizle
if (FindObjectOfType<TrainingAnalyticsUI>())
    FindObjectOfType<TrainingAnalyticsUI>().ResetTracking();

for (int ep = 1; ep <= maxEpisode; ep++)
{
    Random.InitState(ep);
    currentEpisodeIndex = ep;
    env.Reset();
    // Fazlara gore epsilon yonetimi
    if (targetScript != null)
    {
        if (targetScript.curriculumEnabled)
        {
            int currentPhase = targetScript.currentPhase;

            if (currentPhase != lastPhaseTracker)
            {
                if (currentPhase == 1)
                {
                    // Faz 1: Hızlı (1.0 -> 0.05, Decay 0.992)
                    epsilonDecay = 0.9925f;
                }
                else if (currentPhase == 2)
                {
                    // Faz 2: Orta (Baslangic 0.8, Decay 0.995)
                    epsilon = 0.8f;
                    epsilonDecay = 0.997f;
                    print($"<color=yellow>PHASE 2 START: Epsilon
reset to 0.8</color>");
                }
                else if (currentPhase == 3)
                {
                    // Faz 3: Yavas (Baslangic 0.6, Decay 0.999)
                    epsilon = 0.6f;
                    epsilonDecay = 0.9991f;
                    print($"<color=red>PHASE 3 START: Epsilon
reset to 0.6</color>");
                }
                lastPhaseTracker = currentPhase;
            }
        }
        else
        {
            // Mufredat Kapalı
            if (lastPhaseTracker != 0)
            {
                epsilonDecay = 0.9992f;
                lastPhaseTracker = 0;
            }
        }
    }

    float[] state = GetObservation();
    bool done = false;
    float totalReward = 0;
    int steps = 0;

    currentEpisodeReward = 0;
    currentEpisodeSteps = 0;

```

```

// Bolum istatistiklerini sifirlama
System.Array.Clear(actionCounts, 0, actionCounts.Length);
int episodeRandomActionCount = 0;
int episodeBombs = 0;
int episodeDangerSteps = 0;

while (!done && steps < maxStepsPerEpisode)
{
    int action = 0;
    float[] qValues = mlp.Forward(state);

    bool isRandomMove = false;
    // Karar mekanizması
    if (useSafetyRule && pathfinder.IsInDanger(env.gridX,
env.gridY))
    {
        int safeMove = pathfinder.GetSafeMove(env.gridX,
env.gridY);
        action = (safeMove != -1) ? safeMove :
GetActionFromQ(qValues);
    }
    else
    {
        if (Random.Range(0f, 1f) < epsilon)
        {
            action = GetRandomValidAction();
            isRandomMove = true;
        }
        else
        {
            action = GetActionFromQ(qValues);
        }
    }

    if (action < actionCounts.Length) actionCounts[action]++;
    if (isRandomMove) episodeRandomActionCount++;
    if (action == 4 && env.agentBombActive == false)
episodeBombs++;
    if (pathfinder.IsInDanger(env.gridX, env.gridY))
episodeDangerSteps++;

    (float reward, bool terminated) = env.Step(action);
    done = terminated;

    float[] nextState = GetObservation();
    replayBuffer.Add(state, action, reward, nextState, done);

    if (replayBuffer.Count > batchSize)
    {
        TrainNetwork();
    }

    state = nextState;
    totalReward += reward;
    steps++;

    currentEpisodeReward = totalReward;
    currentEpisodeSteps = steps;

    yield return null;
}

string resultReason = "TIMEOUT";

```

```

        string dType = "None";

        if (env.kill) { resultReason = "WIN"; winCount++; }
        else if (!env.isAlive)
        {
            resultReason = "DEATH";
            deathCount++;
            dType = env.deathType.ToString(); // Suicide veya
KilledByTarget
        }
        else { timeoutCount++; }

        lastOutcome = resultReason;
        lastDeathType = dType;
        lastBombsDropped = episodeBombs;
        lastSafetyDangerSteps = episodeDangerSteps;
        lastExplorationRate = (steps > 0) ?
(float)episodeRandomActionCount / steps : 0;

        if (epsilon > minEpsilon)
        {
            epsilon = Mathf.Max(minEpsilon, epsilon * epsilonDecay);
        }

        string actionReport = $"Actions => Y:{actionCounts[0]}
A:{actionCounts[1]} S:{actionCounts[2]} Sol:{actionCounts[3]}
B:{actionCounts[4]} WAIT:{actionCounts[5]}";

        print($"Ep: {ep} | {resultReason} | R: {totalReward:F1} | Eps:
{epsilon:F3} (Decay:{epsilonDecay:F4}) | Ph:{targetScript?.currentPhase} |
{actionReport}");

        if (ep % 20 == 0)
        {
            print($"20 Ep Durum: Wins:{winCount} Deaths:{deathCount}
Timeouts:{timeoutCount}");
            winCount = 0; deathCount = 0; timeoutCount = 0;
            SaveNetwork();
        }
    }

    // Egitim bittiginde son episode'u zorla yazdir
    if (FindObjectOfType<TrainingAnalyticsUI>())

FindObjectOfType<TrainingAnalyticsUI>().LogEpisodeData(currentEpisodeIndex)
;

    SaveNetwork();
    isTraining = false;
}
public IEnumerator AjanTestingLoop(int episodeCount)
{
    isTraining = false;

    // Egitim baslarken UI sayacini temizle
    if (FindObjectOfType<TrainingAnalyticsUI>())
        FindObjectOfType<TrainingAnalyticsUI>().ResetTracking();

    float testEpsilon = 0f;

    for (int ep = 1; ep <= episodeCount; ep++)
    {
        currentEpisodeIndex = ep;

```

```

env.Reset();

bool done = false;
int steps = 0;
float totalReward = 0;
int episodeBombs = 0;
int episodeDangerSteps = 0;

while (!done && steps < 400) // Test için max step
{
    float[] state = GetObservation();
    float[] qValues = mlp.Forward(state);
    int action = GetActionFromQ(qValues);

    // Güvenlik kuralı aktifse ve tehlike varsa ez
    if (useSafetyRule && pathfinder.IsInDanger(env.gridX,
env.gridY))
    {
        int safeMove = pathfinder.GetSafeMove(env.gridX,
env.gridY);
        if (safeMove != -1) action = safeMove;
    }

    // Metrik takibi
    if (action == 4 && env.agentBombActive == false)
episodeBombs++;
    if (pathfinder.IsInDanger(env.gridX, env.gridY))
episodeDangerSteps++;

    (float r, bool t) = env.Step(action);

    totalReward += r;
    done = t;
    steps++;

    currentEpisodeReward = totalReward;
    currentEpisodeSteps = steps;

    yield return null;
}

string resultReason = "TIMEOUT";
string dType = "None";
if (env.kill) resultReason = "WIN";
else if (!env.isAlive) { resultReason = "DEATH"; dType =
env.deathType.ToString(); }

lastOutcome = resultReason;
lastDeathType = dType;
lastBombsDropped = episodeBombs;
lastSafetyDangerSteps = episodeDangerSteps;
lastExplorationRate = 0;

yield return new WaitForSeconds(0.01f);
}

// Egitim bittiginde son episode'u yazdır
if (FindObjectOfType<TrainingAnalyticsUI>())

FindObjectOfType<TrainingAnalyticsUI>().LogEpisodeData(currentEpisodeIndex)
;
}
void TrainNetwork()

```



```

{
    List<Experience> batch = replayBuffer.Sample(batchSize);
    foreach (var exp in batch)
    {
        float targetQ = exp.reward;
        if (!exp.done)
        {
            float[] nextQValues = mlp.Forward(exp.nextState);
            float maxQ = nextQValues.Max();

            // NaN kontrolü
            if (float.IsNaN(maxQ) || float.IsInfinity(maxQ))
            {
                Debug.LogWarning("NaN in maxQ! Using 0.");
                maxQ = 0f;
            }

            targetQ += gamma * maxQ;
        }

        targetQ = Mathf.Clamp(targetQ, -50f, 50f);

        float[] currentQValues = mlp.Forward(exp.state);
        float[] targetQVector = (float[])currentQValues.Clone();
        targetQVector[exp.action] = targetQ;
        mlp.Train(exp.state, targetQVector);
    }
}

int GetActionFromQ(float[] qValues)
{
    // 1. Bomba maskeleme
    if (env.bombActive)
    {
        qValues[4] = float.NegativeInfinity;
    }
    bool up = IsValidMove(env.gridX, env.gridY + 1);
    bool down = IsValidMove(env.gridX, env.gridY - 1);
    bool right = IsValidMove(env.gridX + 1, env.gridY);
    bool left = IsValidMove(env.gridX - 1, env.gridY);

    // 2. Duvar maskeleme
    if (!up) qValues[0] = float.NegativeInfinity;
    if (!down) qValues[1] = float.NegativeInfinity;
    if (!right) qValues[2] = float.NegativeInfinity;
    if (!left) qValues[3] = float.NegativeInfinity;

    float maxVal = float.MinValue;
    int bestAction = -1;

    for (int i = 0; i < qValues.Length; i++)
    {
        if (qValues[i] > maxVal && qValues[i] !=
float.NegativeInfinity)
        {
            maxVal = qValues[i];
            bestAction = i;
        }
    }

    return bestAction;
}

```

```

int GetRandomValidAction()
{
    List<int> validActions = new List<int>();

    bool up = IsValidMove(env.gridX, env.gridY + 1);
    bool down = IsValidMove(env.gridX, env.gridY - 1);
    bool right = IsValidMove(env.gridX + 1, env.gridY);
    bool left = IsValidMove(env.gridX - 1, env.gridY);

    if (up) validActions.Add(0);
    if (down) validActions.Add(1);
    if (right) validActions.Add(2);
    if (left) validActions.Add(3);

    if (!env.bombActive)
        validActions.Add(4);

    validActions.Add(5);

    if (validActions.Count > 0)
        return validActions[Random.Range(0, validActions.Count)];

    Debug.LogWarning("Geçerli action yok.");
    return -1;
}

bool IsValidMove(int x, int y)
{
    if (x < 0 || x >= env.width || y < 0 || y >= env.height)
        return false;

    if (env.map[x, y] != 0)
        return false;

    return true;
}

void SaveNetwork()
{
    string path = Path.Combine(Application.persistentDataPath,
currentSaveFileName);
    mlp.SaveModel(path);
}

void LoadNetwork()
{
    string path = Path.Combine(Application.persistentDataPath,
currentSaveFileName);

    if (File.Exists(path))
    {
        mlp.LoadModel(path);
        print($"<color=green>Model Loaded:
{currentSaveFileName}</color>");
    }
    else
    {
        mlp = new SimpleMLP(inputSize, hiddenLayerSize, env.numActions,
learningRate);
        print($"<color=yellow>New Model Created:
{currentSaveFileName}</color>");
    }
}

```

```

private void OnDrawGizmos()
{
    if (env == null) return;

    float cellSize = env.cellSize;
    Vector3 agentPos = Application.isPlaying ? transform.position :
transform.position;

    Vector2Int[] currentPattern = observationPattern;
    if (currentPattern == null || currentPattern.Length == 0) return;

    foreach (Vector2Int offset in currentPattern)
    {
        Vector3 drawPos = agentPos + new Vector3(offset.x * cellSize,
offset.y * cellSize, 0);

        float distanceFromCenter = Mathf.Sqrt(offset.x * offset.x +
offset.y * offset.y);
        float alpha = 0.5f - (distanceFromCenter * 0.15f);
        if (alpha < 0.1f) alpha = 0.1f;

        Gizmos.color = new Color(1, 1, 1, alpha);
        Gizmos.DrawWireCube(drawPos, Vector3.one * cellSize);

        if (Application.isPlaying)
        {
            Collider2D hit = Physics2D.OverlapBox(drawPos, Vector2.one
* (cellSize * 0.8f), 0);
            if (hit != null)
            {
                if (hit.CompareTag("wall"))
                {
                    Gizmos.color = Color.black;
                    Gizmos.DrawCube(drawPos, Vector3.one * cellSize *
0.9f);
                }
                else if (hit.CompareTag("breakable"))
                {
                    Gizmos.color = new Color(0.5f, 0.5f, 0.5f, 0.4f);
                    Gizmos.DrawCube(drawPos, Vector3.one * cellSize *
0.9f);
                }
                else if (hit.CompareTag("target"))
                {
                    Gizmos.color = Color.green;
                    Gizmos.DrawCube(drawPos, Vector3.one * cellSize *
0.5f);
                }
                else if (hit.CompareTag("bomb"))
                {
                    Gizmos.color = Color.red;
                    Gizmos.DrawSphere(drawPos, cellSize * 0.3f);
                }
            }
        }
    }
}

if (Application.isPlaying && env != null)
{
    Vector3 targetWorldPos = new Vector3(
        env.targetX * cellSize,
        env.targetY * cellSize,
        0);
}

```

```

        Gizmos.color = Color.cyan;
        Gizmos.DrawLine(agentPos, targetWorldPos);
    }
}

```

QBombENV_sc.cs

Oyun dünyasının fiziksel kurallarını ve ızgara (grid) yapısını tanımlar; ajan ve hedefin hareketlerini denetler, ödül (reward) mekanizmasını işletir ve her yeni bölüm (episode) başladığında haritayı sıfırlayarak simülasyonu yeniden başlatır.

```

using System.Collections.Generic;
using UnityEngine;

```

```

public class QBombENV_sc : MonoBehaviour
{
    public GameObject wall;
    public GameObject breakable_wall;
    public GameObject bomb;
    public GameObject agentObject;
    public GameObject targetObject;

    [Header("Env Bilgisi")]
    public float cellSize = 2.0f;
    public int width = 9, height = 7;
    public int numActions = 6;

    [Header("State Bilgisi")]
    public int gridX;
    public int gridY;
    public bool isAlive;
    public bool agentBombActive;
    public bool targetBombActive;

    public bool[,] dangerMap;

    public int targetX, targetY;
    public bool kill;
    private bool terminated;

    public int[,] map; // 0:zemin, 1:breakable, 2:unbreakable

    // Istatistik için
    public int totalBreakableWalls = 0;
    public int wallsBroken = 0;

    private Pathfinder pathfinder;

    private List<GameObject> activeBombs = new List<GameObject>(); //
    Aktif tüm bombalar
    public enum DeathType { None, Suicide, KilledByTarget }
    public DeathType deathType = DeathType.None;

    private int consecutiveWaitCount = 0;

    public bool bombActive
    {
        get { return agentBombActive || targetBombActive; }
        set { agentBombActive = value; }
    }

    private string deathReason = "";
}

```

```

public void LogDeath(string reason)
{
    deathReason = reason;
}

private void Start()
{
    if (agentObject != null)
    {
        pathfinder = agentObject.GetComponent<Pathfinder>();
        if (pathfinder == null) pathfinder =
agentObject.AddComponent<Pathfinder>();
    }
    CreateGrid();
    ResetAgentAndTarget();
}

public void CreateGrid()
{
    map = new int[width, height];
    dangerMap = new bool[width, height];

    totalBreakableWalls = 0;
    wallsBroken = 0;

    for (int x = 0; x < width; x++)
    {
        for (int y = 0; y < height; y++)
        {
            Vector3 pos = new Vector3(x * cellSize, y * cellSize, 0);
            dangerMap[x, y] = false;

            if (x == 0 || y == 0 || x == width - 1 || y == height - 1
|| (x % 2 == 0 && y % 2 == 0))
            {
                map[x, y] = 2;
                Instantiate(wall, pos, Quaternion.identity);
            }
            else
            {
                // Başlangıç (1,1) ve etrafını, Hedef (son-2, son-2)
ve etrafını boş bırak
                if ((x != 1 || y != 1) && (x != 1 || y != 2) && (x !=
2 || y != 1) && (x != width - 2 || y != height - 2)
&& (x != width - 2 || y != height - 3) && (x !=
width - 3 || y != height - 2))
                {
                    if (Random.Range(0, 2) == 0) // %50 ihtimalle
kirilabilir duvar
                    {
                        map[x, y] = 1;
                        totalBreakableWalls++;
                        Instantiate(breakable_wall, pos,
Quaternion.identity);
                    }
                    else map[x, y] = 0;
                }
                else map[x, y] = 0;
            }
        }
    }
}

```

```

// Ajan ve Targeti sifirla
void ResetAgentAndTarget()
{
    gridX = 1;
    gridY = 1;
    isAlive = true;
    deathType = DeathType.None; // Sifirla

    if (agentObject != null)
        agentObject.transform.position = new Vector3(gridX * cellSize,
gridY * cellSize, 0);

    kill = false;
    consecutiveWaitCount = 0;

    bool randomPos = false;
    if (targetObject != null)
    {
        CurriculumTarget ct =
targetObject.GetComponent<CurriculumTarget>();
        if (ct != null && ct.currentPhase == 1) randomPos = true;
    }

    if (randomPos) SetRandomTargetPosition(); // Faz 1: Rastgele
    else { targetX = width - 2; targetY = height - 2; } // Faz 2 ve 3:
Sabit

    if (targetObject != null)
        targetObject.transform.position = new Vector3(targetX *
cellSize, targetY * cellSize, 0);
    }

    void SetRandomTargetPosition()
    {
        int attempts = 30;
        while (attempts > 0)
        {
            int rx = Random.Range(1, width - 1);
            int ry = Random.Range(1, height - 1);
            if (Mathf.Abs(rx - gridX) + Mathf.Abs(ry - gridY) < 3)
{ attempts--; continue; }
            if (map[rx, ry] == 0) { targetX = rx; targetY = ry; return; }
            attempts--;
        }
        targetX = width - 2;
        targetY = height - 2;
    }

    // Bomba koyma fonksiyonu - owner bilgisiyle
    public GameObject PlaceBomb(int x, int y, SimpleBomb.BombOwner owner)
    {
        GameObject bombObj = Instantiate(bomb, new Vector3(x * cellSize, y
* cellSize, 0), Quaternion.identity);
        SimpleBomb bombScript = bombObj.GetComponent<SimpleBomb>();

        if (bombScript != null) bombScript.SetOwner(owner);
        activeBombs.Add(bombObj);
        MarkBombDanger(x, y, true);

        if (owner == SimpleBomb.BombOwner.Agent) agentBombActive = true;
        else if (owner == SimpleBomb.BombOwner.Target) targetBombActive =
true;

```

```

        return bombObj;
    }

    public void MarkBombDanger(int x, int y, bool active)
    {
        Vector2Int[] dirs = {
            new Vector2Int(0, 0),
            new Vector2Int(1, 0),
            new Vector2Int(-1, 0),
            new Vector2Int(0, 1),
            new Vector2Int(0, -1)
        };

        foreach (var d in dirs)
        {
            int nx = x + d.x;
            int ny = y + d.y;
            if (nx >= 0 && nx < width && ny >= 0 && ny < height)
            {
                dangerMap[nx, ny] = active;
            }
        }
    }

    public void WallDestroyed()
    {
        wallsBroken++;
    }

    public (float reward, bool done) Step(int action)
    {
        float reward = -0.02f;
        int newX = gridX;
        int newY = gridY;

        if (!isAlive)
        {
            if (deathType == DeathType.Suicide)
            {
                Debug.Log("---- SUICIDE (-300) ----");
                return (-300f, true);
            }
            else
            {
                Debug.Log("---- KILLED BY TARGET (-100) ----");
                return (-100f, true);
            }
        }
        if (kill)
        {
            Debug.Log("---- TARGET ELIMINATED (+200) ----");
            return (200f, true);
        }
        // --- BEKLEME (5) ---
        if (action == 5)
        {
            if (!agentBombActive) reward -= 0.2f;
            consecutiveWaitCount++;
            if (consecutiveWaitCount > 3) reward -= 0.15f *
(consecutiveWaitCount - 3);
        }
        else consecutiveWaitCount = 0;
    }

```

```

float oldDistanceToTarget = float.MaxValue;
if (pathfinder != null && targetX != -1)
    oldDistanceToTarget = pathfinder.GetDistanceToTarget(gridX,
gridY, targetX, targetY);

// --- HAREKETLER (0-3) ---
if (action == 0) newY += 1; // Yukari
else if (action == 1) newY -= 1; // Asagi
else if (action == 2) newX += 1; // Sag
else if (action == 3) newX -= 1; // Sol

// --- BOMBA KOYMA (4) ---
else if (action == 4)
{
    if (!agentBombActive)
    {
        PlaceBomb(gridX, gridY, SimpleBomb.BombOwner.Agent);
        bool threatensTarget = IsTargetInBlastRange(gridX, gridY);
        int broken = stratejiControl(gridX, gridY);

        if (!threatensTarget)
        {
            if (broken == 0)
            {
                // Hedef menzilde degil ve duvar kirilmadi
                float dist = Mathf.Abs(targetX - gridX) +
Mathf.Abs(targetY - gridY);
                if (dist < 3.0f) reward -= 0.1f; // Hedef yakinda,
belki gelir stratejik
                else if (dist == 3.0f) reward -= 0.5f; // Hedef
gelebilir ama daha dusuk ihtimal
                else reward -= 4.0f; // Hedef uzakta, etraf bos
            }
            else reward += (broken * 6f); // duvar kiriyorsa
        }
        else reward += 20.0f; // Hedef menzildeyse
    }
    else reward -= 5f; // Bomba varken
}

// Targete yakinlasma uzaklasma
if (pathfinder != null && targetX != -1 && action != 4 &&
action != 5)
{
    float newDistanceToTarget =
pathfinder.GetDistanceToTarget(gridX, gridY, targetX, targetY);

    if (newDistanceToTarget < oldDistanceToTarget &&
newDistanceToTarget != float.MaxValue)
        reward += 0.1f;
    else
        reward -= 0.15f;
}

// Duvar kontrolu
if (action >= 0 && action <= 3)
{
    bool collision = false;
    if (newX < 0 || newX >= width || newY < 0 || newY >= height)
collision = true;
    else if (map[newX, newY] != 0) collision = true;
}

```



```

        if (collision)
        {
            reward -= 0.5f;
            newX = gridX; newY = gridY;
        }
        else
        {
            gridX = newX;
            gridY = newY;
        }
    }
    if (pathfinder.IsInDanger(gridX, gridY)) reward -= 0.05f;

    if (agentObject != null)
        agentObject.transform.position = new Vector3(gridX * cellSize,
gridY * cellSize, 0);

    // Tum bombalari guncelle
    UpdateAllBombs();

    // Target hareketi
    if (targetObject != null)
    {
        CurriculumTarget ct =
targetObject.GetComponent<CurriculumTarget>();
        if (ct != null)
        {
            ct.OnTargetStep();
        }
    }
    return (reward, terminated);
}

void UpdateAllBombs()
{
    // Null bombalari temizle
    activeBombs.RemoveAll(b => b == null);

    // Her bombayı güncelle
    foreach (GameObject bombObj in activeBombs)
    {
        if (bombObj != null)
        {
            SimpleBomb bombScript = bombObj.GetComponent<SimpleBomb>();
            if (bombScript != null)
            {
                bombScript.OnStep();
            }
        }
    }
}

bool IsTargetInBlastRange(int bombX, int bombY)
{
    if (targetX == -1 || targetY == -1) return false;

    Vector2Int[] dirs = { new Vector2Int(0, 0), new Vector2Int(1, 0),
new Vector2Int(-1, 0), new Vector2Int(0, 1), new Vector2Int(0, -1) };
    foreach (var d in dirs)
    {
        int nx = bombX + d.x;
        int ny = bombY + d.y;
    }
}

```

```

        if (nx == targetX && ny == targetY) return true;
    }
    return false;
}

int stratejiControl(int x, int y)
{
    int brokenCount = 0;
    Vector2Int[] dirs = { new Vector2Int(0, 0), new Vector2Int(1, 0),
new Vector2Int(-1, 0), new Vector2Int(0, 1), new Vector2Int(0, -1) };
    foreach (var d in dirs)
    {
        int nx = x + d.x;
        int ny = y + d.y;
        if (nx >= 0 && nx < width && ny >= 0 && ny < height)
        {
            if (map[nx, ny] == 1) brokenCount++;
        }
    }
    return brokenCount;
}

public void Reset()
{
    // Duvarlar ve Bombaları temizle
    foreach (var obj in GameObject.FindGameObjectsWithTag("breakable"))
    Destroy(obj);
    foreach (var obj in GameObject.FindGameObjectsWithTag("wall"))
    Destroy(obj);
    foreach (var obj in GameObject.FindGameObjectsWithTag("bomb"))
    Destroy(obj);

    CreateGrid(); // Haritayı yeniden oluştur
    activeBombs.Clear(); // Temizle

    terminated = false;
    agentBombActive = false;
    targetBombActive = false;

    // Ajan ve Target pozisyonlarını sıfırla
    ResetAgentAndTarget();

    if (targetObject != null)
    {
        CurriculumTarget ct =
targetObject.GetComponent<CurriculumTarget>();
        if (ct != null)
        {
            ct.OnEpisodeEnd();
        }
    }
}
}

```

Pathfinder.cs

A* algoritması benzeri bir mantıkla çalışan yol bulma sistemidir; ajanın hedefe en kısa yolu bulmasını sağladığı gibi, yaklaşan patlamalardan kaçabilmesi için bir "risk haritası" oluşturarak güvenli bölgeleri tespit eder.

```

using System.Collections.Generic;
using System.Linq;
using UnityEngine;

```

```

public class Pathfinder : MonoBehaviour
{
    private QBombENV_sc env;

    public bool showGizmos = true;
    private List<Vector2Int> lastCalculatedPath;
    private Vector2Int? currentTargetCell;
    private List<Vector2Int> escapePath;

    public enum PathType { None, Clear, Breakable }
    public PathType currentPathType = PathType.None;

    [Header("Risk Parametreleri")]
    public float dangerWeight = 50f;
    public float enemyProximityWeight = 15f;
    public float deadEndWeight = 20f;
    public int safetyRadius = 2;

    private float[,] riskMap;

    private class Node
    {
        public int x, y;
        public int gCost;
        public int hCost;
        public float riskCost;
        public Node parent;

        public float fCost { get { return gCost + hCost + riskCost; } }

        public Node(int x, int y)
        {
            this.x = x;
            this.y = y;
            this.riskCost = 0;
        }
    }

    void Start()
    {
        env = FindObjectOfType<QBombENV_sc>();
        if (env != null)
        {
            riskMap = new float[env.width, env.height];
        }
    }

    void Update()
    {
        if (env != null)
        {
            UpdateRiskMap();
        }
    }

    public float GetDistanceToTarget(int currentX, int currentY, int
targetX, int targetY)
    {
        // Duvarsız yol varsa
        currentPathType = PathType.Clear;
        List<Vector2Int> path = FindPath(new Vector2Int(currentX,
currentY), new Vector2Int(targetX, targetY), false);
    }
}

```

```

        // Duvarsız yol yoksa, kirilabilir duvarlı yol dene
        if (path == null)
        {
            currentPathType = PathType.Breakable;
            path = FindPath(new Vector2Int(currentX, currentY), new
Vector2Int(targetX, targetY), true);
        }

        lastCalculatedPath = path;

        if (path != null) return path.Count;

        currentPathType = PathType.None;
        return float.MaxValue;
    }

    public int GetSafeMove(int currentX, int currentY)
    {
        UpdateRiskMap();

        Vector2Int currentPos = new Vector2Int(currentX, currentY);
        Vector2Int targetPos = FindNearestSafeCell(currentPos);

        currentTargetCell = targetPos;

        if (targetPos == currentPos)
        {
            return -1;
        }

        List<Vector2Int> path = FindPath(currentPos, targetPos, false);
        escapePath = path;
        // lastCalculatedPath = path;

        if (path != null && path.Count > 0)
        {
            Vector2Int nextStep = path[0];

            if (nextStep.x > currentX) return 2;
            if (nextStep.x < currentX) return 3;
            if (nextStep.y > currentY) return 0;
            if (nextStep.y < currentY) return 1;
        }

        return GetRandomMove(currentX, currentY);
    }

    private void UpdateRiskMap()
    {
        if (env == null || riskMap == null) return;

        for (int x = 0; x < env.width; x++)
            for (int y = 0; y < env.height; y++)
                riskMap[x, y] = 0;

        // Bomba tehlikesi
        for (int x = 0; x < env.width; x++)
            for (int y = 0; y < env.height; y++)
                if (env.dangerMap[x, y]) riskMap[x, y] += dangerWeight;

        // Rakibe yakınlık
        if (env.targetX >= 0 && env.targetY >= 0)

```

```

    {
        for (int x = 0; x < env.width; x++)
        {
            for (int y = 0; y < env.height; y++)
            {
                int dist = Mathf.Abs(x - env.targetX) + Mathf.Abs(y -
env.targetY);
                if (dist <= safetyRadius)
                {
                    float proximityRisk = enemyProximityWeight * (1f -
dist / (float)safetyRadius);
                    riskMap[x, y] += proximityRisk;
                }
            }
        }

        // Kör koridor
        for (int x = 0; x < env.width; x++)
        {
            for (int y = 0; y < env.height; y++)
            {
                if (IsWalkable(x, y, false))
                {
                    int escapeRoutes = CountEscapeRoutes(x, y);
                    if (escapeRoutes <= 1) riskMap[x, y] += deadEndWeight
* (2 - escapeRoutes);
                }
            }
        }

        public int CountEscapeRoutes(int x, int y)
        {
            int count = 0;
            Vector2Int[] directions = { new Vector2Int(0, 1), new Vector2Int(0,
-1), new Vector2Int(1, 0), new Vector2Int(-1, 0) };

            foreach (var dir in directions)
            {
                int nx = x + dir.x;
                int ny = y + dir.y;
                if (IsInBounds(nx, ny) && IsWalkable(nx, ny, false)) count++;
            }
            return count;
        }

        private List<Vector2Int> FindPath(Vector2Int startPos, Vector2Int
targetPos, bool allowBreakables)
        {
            List<Node> openList = new List<Node>();
            HashSet<string> closedList = new HashSet<string>();

            Node startNode = new Node(startPos.x, startPos.y);
            Node targetNode = new Node(targetPos.x, targetPos.y);

            openList.Add(startNode);

            while (openList.Count > 0)
            {
                Node currentNode = openList.OrderBy(n => n.fCost).ThenBy(n =>
n.hCost).First();

```

```

        if (currentNode.x == targetNode.x && currentNode.y ==
targetNode.y)
        {
            return RetracePath(startNode, currentNode);
        }

        openList.Remove(currentNode);
        closedList.Add($"{currentNode.x},{currentNode.y}");

        foreach (Vector2Int neighborPos in GetNeighbors(currentNode.x,
currentNode.y))
        {
            if (closedList.Contains($"{neighborPos.x},{neighborPos.y}")
||
allowBreakables))
                !IsWalkable(neighborPos.x, neighborPos.y,
continue;

                int newGCost = currentNode.gCost + 1;

                if (allowBreakables && env.map[neighborPos.x,
neighborPos.y] == 1) newGCost += 2;

                float newRiskCost = riskMap[neighborPos.x, neighborPos.y];

                Node neighborNode = openList.FirstOrDefault(n => n.x ==
neighborPos.x && n.y == neighborPos.y);
                bool isNewNode = neighborNode == null;

                if (isNewNode || newGCost < neighborNode.gCost)
                {
                    if (isNewNode)
                    {
neighborPos.y);
                        neighborNode = new Node(neighborPos.x,
openList.Add(neighborNode);
                    }
                    neighborNode.gCost = newGCost;
                    neighborNode.hCost = Mathf.Abs(neighborNode.x -
targetNode.x) + Mathf.Abs(neighborNode.y - targetNode.y);
                    neighborNode.riskCost = newRiskCost;
                    neighborNode.parent = currentNode;
                }
            }
        }
        return null;
    }
    private List<Vector2Int> RetracePath(Node startNode, Node endNode)
    {
        List<Vector2Int> path = new List<Vector2Int>();
        Node currentNode = endNode;
        while (currentNode != startNode)
        {
            path.Add(new Vector2Int(currentNode.x, currentNode.y));
            currentNode = currentNode.parent;
        }
        path.Reverse();
        return path;
    }

    private List<Vector2Int> GetNeighbors(int x, int y)
    {
        List<Vector2Int> neighbors = new List<Vector2Int>();

```

```

        if (IsInBounds(x + 1, y)) neighbors.Add(new Vector2Int(x + 1, y));
        if (IsInBounds(x - 1, y)) neighbors.Add(new Vector2Int(x - 1, y));
        if (IsInBounds(x, y + 1)) neighbors.Add(new Vector2Int(x, y + 1));
        if (IsInBounds(x, y - 1)) neighbors.Add(new Vector2Int(x, y - 1));
        return neighbors;
    }

    private Vector2Int FindNearestSafeCell(Vector2Int currentPos)
    {
        if (IsSafe(currentPos.x, currentPos.y)) return currentPos;

        float minRisk = float.MaxValue;
        Vector2Int bestCell = currentPos;

        for (int distance = 1; distance < Mathf.Max(env.width, env.height);
distance++)
        {
            for (int dx = -distance; dx <= distance; dx++)
            {
                for (int dy = -distance; dy <= distance; dy++)
                {
                    if (Mathf.Abs(dx) + Mathf.Abs(dy) != distance)
continue;

                    int x = currentPos.x + dx;
                    int y = currentPos.y + dy;

                    if (IsInBounds(x, y) && IsWalkable(x, y, false))
                    {
                        float risk = riskMap[x, y];
                        if (!env.dangerMap[x, y] && risk < minRisk)
                        {
                            minRisk = risk;
                            bestCell = new Vector2Int(x, y);
                        }
                    }
                }
            }
            if (bestCell != currentPos) return bestCell;
        }
        return currentPos;
    }

    public bool IsInDanger(int x, int y)
    {
        if (!IsInBounds(x, y)) return false;
        return env.dangerMap[x, y];
    }

    private bool IsSafe(int x, int y) => !IsInDanger(x, y);

    private bool IsWalkable(int x, int y, bool allowBreakables)
    {
        if (!IsInBounds(x, y)) return false;
        if (env.map[x, y] == 0) return true;
        if (allowBreakables && env.map[x, y] == 1) return true;
        return false;
    }

    private bool IsInBounds(int x, int y)
    {
        return x >= 0 && x < env.width && y >= 0 && y < env.height;
    }

```

```

private int GetRandomMove(int currentX, int currentY)
{
    List<int> possibleMoves = new List<int>();
    if (IsWalkable(currentX, currentY + 1, false))
possibleMoves.Add(0);
    if (IsWalkable(currentX, currentY - 1, false))
possibleMoves.Add(1);
    if (IsWalkable(currentX + 1, currentY, false))
possibleMoves.Add(2);
    if (IsWalkable(currentX - 1, currentY, false))
possibleMoves.Add(3);

    if (possibleMoves.Count > 0)
        return possibleMoves[Random.Range(0, possibleMoves.Count)];
    return -1;
}

private void OnDrawGizmos()
{
    if (!showGizmos || env == null) return;

    // Risk haritasi
    if (riskMap != null)
    {
        for (int x = 0; x < env.width; x++)
        {
            for (int y = 0; y < env.height; y++)
            {
                if (riskMap[x, y] > 0)
                {
                    float intensity = Mathf.Clamp01(riskMap[x, y] /
100f);
                    Gizmos.color = new Color(1f, 0f, 0f, intensity *
0.5f);

                    Vector3 pos = new Vector3(
                        x * env.cellSize,
                        y * env.cellSize,
                        0);
                    Gizmos.DrawCube(pos, Vector3.one * env.cellSize *
0.8f);
                }
            }
        }
    }

    // Guvenli hucre
    if (currentTargetCell.HasValue)
    {
        Gizmos.color = Color.yellow;
        Vector3 targetWorldPos = new Vector3(
            currentTargetCell.Value.x * env.cellSize,
            currentTargetCell.Value.y * env.cellSize,
            0);
        Gizmos.DrawWireSphere(targetWorldPos, env.cellSize * 0.4f);
    }

    // Targete yol
    if (lastCalculatedPath != null && lastCalculatedPath.Count > 0)
    {
        // Yol türüne göre renk
        if (currentPathType == PathType.Clear)
            Gizmos.color = Color.green;
    }
}

```



```

else if (currentPathType == PathType.Breakable)
    Gizmos.color = Color.blue;
else
    Gizmos.color = Color.yellow;

Vector3 startPos = transform.position;
Vector3 firstNodePos = new Vector3(
    lastCalculatedPath[0].x * env.cellSize,
    lastCalculatedPath[0].y * env.cellSize,
    0);

Gizmos.DrawLine(startPos, firstNodePos);

for (int i = 0; i < lastCalculatedPath.Count - 1; i++)
{
    Vector3 p1 = new Vector3(
        lastCalculatedPath[i].x * env.cellSize,
        lastCalculatedPath[i].y * env.cellSize,
        0);
    Vector3 p2 = new Vector3(
        lastCalculatedPath[i + 1].x * env.cellSize,
        lastCalculatedPath[i + 1].y * env.cellSize,
        0);

    Gizmos.DrawLine(p1, p2);
    Gizmos.DrawSphere(p1, 0.15f);
}
}

// Kacıs yolu
if (escapePath != null && escapePath.Count > 0)
{
    Gizmos.color = Color.yellow;

    Vector3 startPos = transform.position;
    Vector3 firstNodePos = new Vector3(
        escapePath[0].x * env.cellSize,
        escapePath[0].y * env.cellSize,
        0);

    Gizmos.DrawLine(startPos, firstNodePos);

    for (int i = 0; i < escapePath.Count - 1; i++)
    {
        Vector3 p1 = new Vector3(
            escapePath[i].x * env.cellSize,
            escapePath[i].y * env.cellSize,
            0);
        Vector3 p2 = new Vector3(
            escapePath[i + 1].x * env.cellSize,
            escapePath[i + 1].y * env.cellSize,
            0);

        Gizmos.DrawLine(p1, p2);
        Gizmos.DrawSphere(p1, 0.12f);
    }
}

// Ajan tehlikede
int myX = Mathf.RoundToInt(transform.position.x / env.cellSize);
int myY = Mathf.RoundToInt(transform.position.y / env.cellSize);

if (IsInDanger(myX, myY))

```

```

        {
            Gizmos.color = new Color(1f, 0f, 0f, 0.7f);
            Gizmos.DrawSphere(transform.position, env.cellSize * 0.5f);
        }
    }
}

```

SimpleMLP.cs

Yapay zeka ajanının beynini oluşturan, ReLU aktivasyon fonksiyonu ve momentum destekli ağırlık güncelleme mekanizmalarına sahip, sıfırdan yazılmış çok katmanlı bir sinir ağı (Multi-Layer Perceptron) yapısıdır.

```

using UnityEngine;
using System;
using System.IO;

[System.Serializable]
public class SimpleMLP
{
    public int inputSize;
    public int hiddenSize;
    public int outputSize;
    public float learningRate;
    public float momentum = 0.9f;

    public float weightDecay = 0.0001f;

    public float[,] weightsIH;
    public float[,] weightsHO;
    public float[] biasH;
    public float[] biasO;

    // Momentum hızları
    private float[,] velocityIH;
    private float[,] velocityHO;
    private float[] velocityBiasH;
    private float[] velocityBiasO;

    private float[] hiddenLayerOutput;

    public SimpleMLP(int inp, int hid, int outSize, float lr)
    {
        inputSize = inp;
        hiddenSize = hid;
        outputSize = outSize;
        learningRate = lr;

        weightsIH = new float[inputSize, hiddenSize];
        weightsHO = new float[hiddenSize, outputSize];
        biasH = new float[hiddenSize];
        biasO = new float[outputSize];

        velocityIH = new float[inputSize, hiddenSize];
        velocityHO = new float[hiddenSize, outputSize];
        velocityBiasH = new float[hiddenSize];
        velocityBiasO = new float[outputSize];

        InitializeWeights();
    }
}

```

```

void InitializeWeights()
{
    UnityEngine.Random.InitState((int)System.DateTime.Now.Ticks);
    float limitIH = Mathf.Sqrt(6.0f / (inputSize + hiddenSize));
    float limitHO = Mathf.Sqrt(6.0f / (hiddenSize + outputSize));

    for (int i = 0; i < inputSize; i++)
        for (int j = 0; j < hiddenSize; j++)
            weightsIH[i, j] = UnityEngine.Random.Range(-limitIH,
limitIH);

    for (int i = 0; i < hiddenSize; i++)
        for (int j = 0; j < outputSize; j++)
            weightsHO[i, j] = UnityEngine.Random.Range(-limitHO,
limitHO);
}

public float[] Forward(float[] inputs)
{
    hiddenLayerOutput = new float[hiddenSize];
    float[] outputLayer = new float[outputSize];

    for (int h = 0; h < hiddenSize; h++)
    {
        float sum = 0;
        for (int i = 0; i < inputSize; i++)
            sum += inputs[i] * weightsIH[i, h];
        sum += biasH[h];
        hiddenLayerOutput[h] = ReLU(sum);
    }

    for (int o = 0; o < outputSize; o++)
    {
        float sum = 0;
        for (int h = 0; h < hiddenSize; h++)
            sum += hiddenLayerOutput[h] * weightsHO[h, o];
        sum += biasO[o];
        outputLayer[o] = sum;
    }
    return outputLayer;
}

public void Train(float[] inputs, float[] targetOutputs)
{
    float[] predictedOutputs = Forward(inputs);

    // Output Delta
    float[] outputDeltas = new float[outputSize];
    for (int o = 0; o < outputSize; o++)
    {
        float error = targetOutputs[o] - predictedOutputs[o];
        outputDeltas[o] = Mathf.Clamp(error, -100f, 100f);
    }

    // Hidden Delta
    float[] hiddenDeltas = new float[hiddenSize];
    for (int h = 0; h < hiddenSize; h++)
    {
        float error = 0;
        for (int o = 0; o < outputSize; o++)
            error += outputDeltas[o] * weightsHO[h, o];
        hiddenDeltas[h] = error * ReLUDerivative(hiddenLayerOutput[h]);
    }
}

```

```

    }

    // Hidden -> Output
    for (int h = 0; h < hiddenSize; h++)
    {
        for (int o = 0; o < outputSize; o++)
        {
            float gradient = outputDeltas[o] * hiddenLayerOutput[h];

            // Weight Decay (-weightDecay * currentWeight)
            // Ağırlığın asiri buyumesini engellemek için
            velocityHO[h, o] = (momentum * velocityHO[h, o]) +
(learningRate * gradient) - (learningRate * weightDecay * weightsHO[h, o]);
            weightsHO[h, o] += velocityHO[h, o];
        }
    }

    // Output Bias
    for (int o = 0; o < outputSize; o++)
    {
        velocityBiasO[o] = (momentum * velocityBiasO[o]) +
(learningRate * outputDeltas[o]);
        biasO[o] += velocityBiasO[o];
    }

    // Input -> Hidden
    for (int i = 0; i < inputSize; i++)
    {
        for (int h = 0; h < hiddenSize; h++)
        {
            float gradient = hiddenDeltas[h] * inputs[i];

            // Weight Decay
            velocityIH[i, h] = (momentum * velocityIH[i, h]) +
(learningRate * gradient) - (learningRate * weightDecay * weightsIH[i, h]);
            weightsIH[i, h] += velocityIH[i, h];
        }
    }

    // Hidden Bias
    for (int h = 0; h < hiddenSize; h++)
    {
        velocityBiasH[h] = (momentum * velocityBiasH[h]) +
(learningRate * hiddenDeltas[h]);
        biasH[h] += velocityBiasH[h];
    }
}

float ReLU(float x) => Math.Max(0, x);
float ReLUDerivative(float x) => x > 0 ? 1 : 0;

[System.Serializable]
private class ModelData
{
    public int inputSize, hiddenSize, outputSize;
    public float[] flatWeightsIH;
    public float[] flatWeightsHO;
    public float[] biasH;
    public float[] biasO;
}

public void SaveModel(string path)
{

```

```

        ModelData data = new ModelData();
        data.inputSize = inputSize; data.hiddenSize = hiddenSize;
        data.outputSize = outputSize;
        data.biasH = biasH; data.bias0 = bias0;

        data.flatWeightsIH = new float[inputSize * hiddenSize];
        for (int i = 0; i < inputSize; i++) for (int j = 0; j < hiddenSize;
j++) data.flatWeightsIH[i * hiddenSize + j] = weightsIH[i, j];

        data.flatWeightsHO = new float[hiddenSize * outputSize];
        for (int i = 0; i < hiddenSize; i++) for (int j = 0; j <
outputSize; j++) data.flatWeightsHO[i * outputSize + j] = weightsHO[i, j];

        File.WriteAllText(path, JsonUtility.ToJson(data));
        Debug.Log("Model saved: " + path);
    }

    public void LoadModel(string path)
    {
        if (!File.Exists(path)) return;
        ModelData data =
JsonUtility.FromJson<ModelData>(File.ReadAllText(path));

        if (data.inputSize != inputSize || data.hiddenSize != hiddenSize
|| data.outputSize != outputSize) return;

        biasH = data.biasH; bias0 = data.bias0;
        for (int i = 0; i < inputSize; i++) for (int j = 0; j < hiddenSize;
j++) weightsIH[i, j] = data.flatWeightsIH[i * hiddenSize + j];
        for (int i = 0; i < hiddenSize; i++) for (int j = 0; j <
outputSize; j++) weightsHO[i, j] = data.flatWeightsHO[i * outputSize + j];

        // Velocity sıfırlama
        Array.Clear(velocityIH, 0, velocityIH.Length);
        Array.Clear(velocityHO, 0, velocityHO.Length);
        Array.Clear(velocityBiasH, 0, velocityBiasH.Length);
        Array.Clear(velocityBias0, 0, velocityBias0.Length);
    }
}

```

ExperienceReplay.cs

Ajanın geçmişteki deneyimlerini (durum, aksiyon, ödül, sonraki durum) hafızada tutan bir "Deneyim Tekrarı" (Replay Buffer) sistemidir; bu sayede ajan, sadece o anki adımından değil, hafızasındaki geçmiş tecrübelerden rastgele örneklemeler (batch) alarak daha istikrarlı bir öğrenme gerçekleştirir.

```

using System.Collections.Generic;
using UnityEngine;

public class Experience
{
    public float[] state;
    public int action;
    public float reward;
    public float[] nextState;
    public bool done;

    public Experience(float[] s, int a, float r, float[] ns, bool d)
    {
        state = s; action = a; reward = r; nextState = ns; done = d;
    }
}

```

```

}

public class ReplayBuffer
{
    private List<Experience> buffer = new List<Experience>();
    private int capacity;

    public ReplayBuffer(int maxCapacity)
    {
        capacity = maxCapacity;
    }

    public void Add(float[] s, int a, float r, float[] ns, bool d)
    {
        if (buffer.Count >= capacity)
        {
            buffer.RemoveAt(0);
        }
        buffer.Add(new Experience(s, a, r, ns, d));
    }

    public List<Experience> Sample(int batchSize)
    {
        List<Experience> batch = new List<Experience>();
        int count = Mathf.Min(batchSize, buffer.Count);

        for (int i = 0; i < count; i++)
        {
            batch.Add(buffer[Random.Range(0, buffer.Count)]);
        }
        return batch;
    }

    public int Count => buffer.Count;
}

```

SimpleBomb.cs

Bomba nesnesinin mantığını yönetir; belirli bir adım (tur) sayısından sonra patlayarak çevresindeki kırılabilir duvarları yok eder ve eğer menzilineyse ajanı veya hedefi oyun dışı bırakır.

```

using UnityEngine;

public class SimpleBomb : MonoBehaviour
{
    private QBombENV_sc env;
    private float cellSize;
    public GameObject patlama;

    public int explosionSteps = 3; // 3 adım sonra patlar
    public int currentStep = 0;

    int bombGridX;
    int bombGridY;

    public enum BombOwner { Agent, Target }
    public BombOwner owner;

    Vector2Int[] dirs = { new Vector2Int(0, 0), new Vector2Int(1, 0), new
Vector2Int(-1, 0), new Vector2Int(0, 1), new Vector2Int(0, -1) };

```

```

private void Start()
{
    env = FindObjectOfType<QBombENV_sc>();
    if (env != null)
    {
        cellSize = env.cellSize;
    }

    bombGridX = Mathf.RoundToInt(transform.position.x / cellSize);
    bombGridY = Mathf.RoundToInt(transform.position.y / cellSize);
}

public void SetOwner(BombOwner bombOwner)
{
    owner = bombOwner;
}

// Ajan her adim attiginda Env tarafindan bu cagilir
public void OnStep()
{
    currentStep++;

    if (currentStep >= explosionSteps)
    {
        Patlat();
    }
}

void Patlat()
{
    DestroyAll(bombGridX, bombGridY);
    if (env != null) env.MarkBombDanger(bombGridX, bombGridY, false);

    Destroy(gameObject);
    // Bomba sahibine gore aktif bomba sayisini azalt
    if (env != null)
    {
        if (owner == BombOwner.Agent)
            env.agentBombActive = false;
        else if (owner == BombOwner.Target)
            env.targetBombActive = false;
    }
}

void DestroyAll(int x, int y)
{
    foreach (var d in dirs)
    {
        int nx = x + d.x;
        int ny = y + d.y;
        if (env != null && nx >= 0 && nx < env.width && ny >= 0 && ny
< env.height)
        {
            if (env.map[nx, ny] == 1) // Kirilabilir duvar
            {
                env.map[nx, ny] = 0; // Harita verisini temizle

                env.WallDestroyed(); // Istatistik icin

                Vector3 wallPos = new Vector3(nx * cellSize, ny *
cellSize, 0);

```



```
[Range(0f, 1f)] public float randomnessFactor = 0.2f;    // Ezberlemeyi engelleme
```

```
[Header("Faz 3: Bomba Parametreleri")]  
public float bombNearAgent = 0.4f;  
public float bombNearWall = 0.6f;
```

```
[Header("Istatistikler")]  
public int totalEpisodes = 0;  
public int currentPhaseEpisodes = 0;
```

```
private int escapeStepsRemaining = 0;  
private Vector2Int safeEscapeTarget;
```

```
public bool curriculumEnabled = true;
```

```
void Start()  
{  
    env = FindObjectOfType<QBombENV_sc>();  
}
```

```
public void SetCurriculumMode(bool enabled)  
{  
    curriculumEnabled = enabled;  
  
    if (!curriculumEnabled)  
    {  
        currentPhase = 3;  
        currentPhaseEpisodes = 0;  
        Debug.Log("Curriculum DISABLED: Locked to Phase 3.");  
    }  
    else  
    {  
        currentPhase = 1;  
        currentPhaseEpisodes = 0;  
        Debug.Log("Curriculum ENABLED: Starting from Phase 1.");  
    }  
}
```

```
public void OnTargetStep()  
{  
    if (env == null || !env.isAlive || env.kill) return;  
  
    if (currentPhase == 1) return;  
    if (currentPhase == 2) { MoveRandomly(); return; }  
  
    if (escapeStepsRemaining > 0)  
    {  
        HandleEscapeSequence();  
        return;  
    }  
  
    if (Random.value < randomnessFactor)  
    {  
        MoveRandomly();  
        return;  
    }  
  
    ExecutePhase3Behavior();  
}
```

```
void ExecutePhase3Behavior()  
{
```

```

// Tehlikeliyse kac
if (env.dangerMap[env.targetX, env.targetY])
{
    MoveToSafestNeighbor();
    return;
}

// Bomba koy
if (!env.targetBombActive)
{
    bool nearAgent = ManhattanDistanceToPlayer() <= 2;
    bool nearWall = IsNearBreakableWall();

    bool shouldPlace = (nearAgent && Random.value < bombNearAgent)

||
                        (nearWall && Random.value < bombNearWall);

    if (shouldPlace)
    {
        Vector2Int escape = FindSafeEscapeCell();
        if (escape.x != -1)
        {
            PlaceBombAndEscape(escape);
            return;
        }
    }

    // Agresiflik seviyesine gore yaklas veya kac
    if (Random.value < aggressionLevel)
        MoveTowardPlayer();
    else
        MoveAwayFromPlayer();
}

void MoveRandomly()
{
    var safeMoves = GetValidSafeMoves();
    if (safeMoves.Count > 0)
        ExecuteMove(safeMoves[Random.Range(0, safeMoves.Count)]);
    else
        MoveToSafestNeighbor();
}

void HandleEscapeSequence()
{
    int dir = GetDirectionTo(safeEscapeTarget);
    if (dir != -1)
        ExecuteMove(dir);
    else
        MoveToSafestNeighbor();

    escapeStepsRemaining--;
}

void MoveTowardPlayer()
{
    MoveToBestNeighbor(pos => -ManhattanDistance(pos, env.gridX,
env.gridY));
}

void MoveAwayFromPlayer()
{

```

```

        MoveToBestNeighbor(pos => ManhattanDistance(pos, env.gridX,
env.gridY));
    }

    void MoveToSafestNeighbor()
    {
        MoveToBestNeighbor(pos =>
        {
            if (!env.dangerMap[pos.x, pos.y]) return 1000; // Güvenliyse
            return -1000; // Tehlikeliyse kotu
        });
    }

    void MoveToBestNeighbor(System.Func<Vector2Int, float> scoreFunc)
    {
        float bestScore = float.MinValue;
        int bestDir = -1;

        for (int dir = 0; dir < 4; dir++)
        {
            Vector2Int next = GetNextPos(dir);
            if (!IsValidPos(next.x, next.y)) continue;

            float score = scoreFunc(next);
            if (score > bestScore) { bestScore = score; bestDir = dir; }
        }

        if (bestDir != -1) ExecuteMove(bestDir);
    }

    List<int> GetValidSafeMoves()
    {
        List<int> moves = new List<int>();
        bool checkSafety = env.targetBombActive;
        for (int i = 0; i < 4; i++)
        {
            Vector2Int n = GetNextPos(i);
            if (IsValidPos(n.x, n.y))
            {
                if (checkSafety && env.dangerMap[n.x, n.y]) continue;
                moves.Add(i);
            }
        }
        return moves;
    }

    void ExecuteMove(int dir)
    {
        Vector2Int n = GetNextPos(dir);
        if (IsValidPos(n.x, n.y))
        {
            env.targetX = n.x;
            env.targetY = n.y;
            transform.position = new Vector3(n.x * env.cellSize, n.y *
env.cellSize, 0);
        }
    }

    bool IsValidPos(int x, int y) => x >= 0 && x < env.width && y >= 0 &&
y < env.height && env.map[x, y] == 0;

    Vector2Int GetNextPos(int dir)

```

```

    {
        int nx = env.targetX, ny = env.targetY;
        switch (dir) { case 0: ny++; break; case 1: ny--; break; case 2:
nx++; break; case 3: nx--; break; }
        return new Vector2Int(nx, ny);
    }

    int ManhattanDistanceToPlayer() => Mathf.Abs(env.targetX - env.gridX)
+   Mathf.Abs(env.targetY - env.gridY);
    int ManhattanDistance(Vector2Int p, int x, int y) => Mathf.Abs(p.x - x)
+   Mathf.Abs(p.y - y);

    bool IsNearBreakableWall()
    {
        for (int i = 0; i < 4; i++)
        {
            Vector2Int n = GetNextPos(i);
            if (n.x >= 0 && n.x < env.width && n.y >= 0 && n.y <
env.height && env.map[n.x, n.y] == 1) return true;
        }
        return false;
    }

    void PlaceBombAndEscape(Vector2Int escapeTarget)
    {
        env.PlaceBomb(env.targetX, env.targetY,
SimpleBomb.BombOwner.Target);
        safeEscapeTarget = escapeTarget;
        escapeStepsRemaining = 4;
    }

    int GetDirectionTo(Vector2Int target)
    {
        int dx = target.x - env.targetX;
        int dy = target.y - env.targetY;
        if (dx > 0 && IsValidMove(2)) return 2;
        if (dx < 0 && IsValidMove(3)) return 3;
        if (dy > 0 && IsValidMove(0)) return 0;
        if (dy < 0 && IsValidMove(1)) return 1;
        return -1;
    }

    bool IsValidMove(int dir) => IsValidPos(GetNextPos(dir).x,
GetNextPos(dir).y);

    Vector2Int FindSafeEscapeCell()
    {
        bool[,] simDanger = new bool[env.width, env.height];
        for (int x = 0; x < env.width; x++) for (int y = 0; y < env.height;
y++) simDanger[x, y] = env.dangerMap[x, y];
        int tx = env.targetX, ty = env.targetY;
        simDanger[tx, ty] = true;
        if (tx + 1 < env.width) simDanger[tx + 1, ty] = true;
        if (tx - 1 >= 0) simDanger[tx - 1, ty] = true;
        if (ty + 1 < env.height) simDanger[tx, ty + 1] = true;
        if (ty - 1 >= 0) simDanger[tx, ty - 1] = true;

        // BFS ile en yakin guvenli hucre bulma
        Queue<Vector2Int> queue = new Queue<Vector2Int>();
        HashSet<Vector2Int> visited = new HashSet<Vector2Int>();
        queue.Enqueue(new Vector2Int(tx, ty));
        visited.Add(new Vector2Int(tx, ty));
    }

```

```

        Vector2Int[] dirs = { Vector2Int.up, Vector2Int.down,
Vector2Int.right, Vector2Int.left };

        while (queue.Count > 0)
        {
            Vector2Int curr = queue.Dequeue();
            if (!simDanger[curr.x, curr.y]) return curr;
            foreach (var d in dirs)
            {
                Vector2Int next = curr + d;
                if (IsValidPos(next.x, next.y) && !visited.Contains(next))
            { visited.Add(next); queue.Enqueue(next); }
            }
            return new Vector2Int(-1, -1);
        }

        public void OnEpisodeEnd()
        {
            escapeStepsRemaining = 0;
            totalEpisodes++;
            currentPhaseEpisodes++;
            if (curriculumEnabled) CheckPhaseTransition();
        }

        void CheckPhaseTransition()
        {
            if (currentPhase == 1 && currentPhaseEpisodes >= phase1Duration)
            { currentPhase = 2; currentPhaseEpisodes = 0; }
            else if (currentPhase == 2 && currentPhaseEpisodes >=
phase2Duration) { currentPhase = 3; currentPhaseEpisodes = 0; }
        }
    }
}

```

ExperimentManager.cs

Farklı eğitim senaryolarını (müfredatlı/müfredatsız veya güvenlik kurallı/kuralsız) otomatize eder; modelleri test etmek için farklı zorluk seviyelerindeki senaryoları sırayla çalıştırarak sonuçları analiz etmenizi sağlar.

```

using UnityEngine;
using UnityEngine.SceneManagement;
using UnityEngine.UI;
using System.Collections;

public class ExperimentManager : MonoBehaviour
{
    [Header("References")]
    public AjanRL agentScript;
    public CurriculumTarget targetScript;
    public TrainingAnalyticsUI uiScript;
    public Text experimentInfoText;

    public enum ExperimentType
    {
        Curriculum_Safety,
        Curriculum_NoSafety,
        NoCurriculum_Safety,
        NoCurriculum_NoSafety
    }

    [Header("Current Status")]

```

```

    public ExperimentType currentExperiment =
ExperimentType.Curriculum_Safety;

    private void Start()
    {
        ApplyExperimentSettings();
    }

    private void Update()
    {
        if (Input.GetKeyDown(KeyCode.E))
        {
            SwitchToNextExperiment();
        }

        // 'T' tusuna basildiginda otomatik test dongusunu baslat
        if (Input.GetKeyDown(KeyCode.T))
        {
            StopAllCoroutines();
            StartCoroutine(RunFullTestSequence());
        }
    }

    // --- TEST SENARYOLARI DONGUSU ---
    IEnumerator RunFullTestSequence()
    {
        Debug.Log($"<color=orange>TEST BASLADI: {currentExperiment} modeli
test ediliyor...</color>");

        // Model ismini uzantisiz al (CSV ismi icin)
        string modelBaseName =
agentScript.currentSaveFileName.Replace(".json", "");

        // Senaryo Tanimlari: Faz, Agresiflik, BombaOlasiligi
        (int phase, float aggro, float bomb)[] scenarios = new (int, float,
float)[]
        {
            (1, 0.0f, 0.0f), // Senaryo 1: Faz 1 (Hedef sabit)
            (2, 0.0f, 0.0f), // Senaryo 2: Faz 2 (Hedef rastgele)
            (3, 0.7f, 0.4f), // Senaryo 3: Faz 3 (Hedef Agresif)
            (3, 0.3f, 0.3f), // Senaryo 4: Faz 3 (dusuk Agresiflik)
            (3, 0.6f, 0.6f), // Senaryo 5: Faz 3 (orta Agresiflik)
            (3, 9.0f, 9.0f) // Senaryo 6: Faz 3 (yuksek Agresiflik/Bomba)
        };

        for (int i = 0; i < scenarios.Length; i++)
        {
            var s = scenarios[i];

            // 1. Hedef Parametrelerini Ayarla
            targetScript.currentPhase = s.phase;
            targetScript.aggressionLevel = s.aggro;
            targetScript.bombNearAgent = s.bomb;
            targetScript.bombNearWall = s.bomb; // Duvar yaninda bomba
koyma da paralel artsin

            // 2. CSV Dosyasini Hazirla
            string scenarioName = $"Test_{modelBaseName}_Scen{i +
1}_Ph{s.phase}_Ag{s.aggro}";
            uiScript.SetCSVFileName(scenarioName + ".csv");
            uiScript.ResetTracking();

            if (experimentInfoText)

```

```

        experimentInfoText.text = $"TESTING Scen {i + 1}/6\nModel:
{modelName}\nPhase: {s.phase}\nAggro: {s.aggro}";

        // 3. 30 Episode Testi Kos
        yield return StartCoroutine(agentScript.AjanTestingLoop(30));

        Debug.Log($"<color=green>Senaryo {i + 1} Tamamlandi:
{scenarioName}</color>");
    }

    Debug.Log("<color=cyan>TUM TEST SENARYOLARI BITTI!</color>");
    if (experimentInfoText) experimentInfoText.text = "TEST
COMPLETED!";
}

void SwitchToNextExperiment()
{
    int current = (int)currentExperiment;
    current = (current + 1) % 4;
    currentExperiment = (ExperimentType)current;

    ApplyExperimentSettings();

    agentScript.StopAllCoroutines();
    FindObjectOfType<QBombENV_sc>().Reset();

    Debug.Log($"<color=cyan>EXPERIMENT SWITCHED TO:
{currentExperiment}</color>");
}

void ApplyExperimentSettings()
{
    string modelName = "";
    bool safety = false;
    bool useCurriculum = false;

    switch (currentExperiment)
    {
        case ExperimentType.Curriculum_Safety:
            modelName = "model_cur_safe.json";
            safety = true;
            useCurriculum = true;
            break;
        case ExperimentType.Curriculum_NoSafety:
            modelName = "model_cur_nosafe.json";
            safety = false;
            useCurriculum = true;
            break;
        case ExperimentType.NoCurriculum_Safety:
            modelName = "model_nocur_safe.json";
            safety = true;
            useCurriculum = false;
            break;
        case ExperimentType.NoCurriculum_NoSafety:
            modelName = "model_nocur_nosafe.json";
            safety = false;
            useCurriculum = false;
            break;
    }

    agentScript.SetExperimentConfig(modelName, safety);
    targetScript.SetCurriculumMode(useCurriculum);
}

```

```

        if (uiScript != null)
            uiScript.SetCSVFileName($"Data_{currentExperiment}.csv");

        if (experimentInfoText)
            experimentInfoText.text = $"EXP: {currentExperiment}\nSafety:
{safety}\nCurriculum: {useCurriculum}";
    }
}

```

TrainingAnalyticsUI.cs

Eğitim sürecini gerçek zamanlı olarak Unity arayüzünde (UI) görselleştirir ve her bölüm (episode) sonunda başarı durumu, kazanılan ödül, kırılan duvar sayısı gibi kritik verileri analiz için bir CSV dosyasına kaydederek performans takibi yapmanıza olanak tanır.

```

using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;
using System.IO;
using System.Linq;

public class TrainingAnalyticsUI : MonoBehaviour
{
    [Header("References")]
    public AjanRL agentScript;
    public CurriculumTarget targetScript;
    public QBombENV_sc envScript;

    [Header("UI Text Fields")]
    public Text episodeText;
    public Text phaseText;
    public Text totalRewardText;
    public Text stepsText;
    public Text epsilonText;

    // CSV Kayıt
    private string csvFilePath;
    private int trackedEpisodeIndex = 0;

    private float tempLastReward = 0;
    private int tempLastSteps = 0;
    private float tempLastEpsilon = 0;

    private int lastWallsBroken = 0;
    private int lastWallsInitial = 0;

    void Start()
    {
        if (!agentScript) agentScript = FindObjectOfType<AjanRL>();
        if (!targetScript) targetScript =
FindObjectOfType<CurriculumTarget>();
        if (!envScript) envScript = FindObjectOfType<QBombENV_sc>();

        // Dosya ismi dinamik belirlenecek
        if (agentScript) trackedEpisodeIndex =
agentScript.currentEpisodeIndex;
    }

    public void ResetTracking()
    {
        trackedEpisodeIndex = 0;
    }
}

```



```

        tempLastReward = 0;
        tempLastSteps = 0;
        Debug.Log("Analiz takibi sifirlandi.");
    }
    public void SetCSVFileName(string fileName)
    {
        string folderPath = Path.Combine(Application.dataPath, "Analysis");
        if (!Directory.Exists(folderPath))
            Directory.CreateDirectory(folderPath);

        csvFilePath = Path.Combine(folderPath, fileName);

        if (!File.Exists(csvFilePath))
        {
            string header =
                "Episode;Outcome;DeathType;Phase;Steps;TotalReward;Epsilon;" +
                "ExplorationRate;WallsInitial;WallsBroken;BombsDropped;SafetyDangerSteps";
            File.WriteAllText(csvFilePath, header + "\n");
        }
        Debug.Log($"CSV Log Hedefi: {fileName}");
    }

    void Update()
    {
        if (agentScript == null) return;

        if (episodeText) episodeText.text = $"Episode:
{agentScript.currentEpisodeIndex} / {agentScript.maxEpisodeIndex}";
        if (phaseText && targetScript) phaseText.text = $"Phase:
{targetScript.currentPhase}";
        if (totalRewardText) totalRewardText.text = $"Reward:
{agentScript.currentEpisodeReward:F1}";
        if (stepsText) stepsText.text = $"Steps:
{agentScript.currentEpisodeSteps}";
        if (epsilonText) epsilonText.text = $"Epsilon:
{agentScript.epsilon:F3}";

        CheckAndLogToCSV();
    }

    void CheckAndLogToCSV()
    {
        if (agentScript.currentEpisodeIndex > trackedEpisodeIndex)
        {
            if (trackedEpisodeIndex > 0)
            {
                LogEpisodeData(trackedEpisodeIndex);
            }
            trackedEpisodeIndex = agentScript.currentEpisodeIndex;
        }

        tempLastReward = agentScript.currentEpisodeReward;
        tempLastSteps = agentScript.currentEpisodeSteps;
        tempLastEpsilon = agentScript.epsilon;

        if (envScript != null)
        {
            lastWallsBroken = envScript.wallsBroken;
            lastWallsInitial = envScript.totalBreakableWalls;
        }
    }
}

```

```

public void LogEpisodeData(int episode)
{
    if (string.IsNullOrEmpty(csvFilePath)) return;

    string outcome = agentScript.lastOutcome;
    string deathType = agentScript.lastDeathType;
    int phase = (targetScript != null) ? targetScript.currentPhase : 0;

    int steps = tempLastSteps;
    float reward = tempLastReward;
    float epsilon = tempLastEpsilon;
    float expRate = agentScript.lastExplorationRate;

    int wInit = lastWallsInitial;
    int wBroken = lastWallsBroken;
    int bombs = agentScript.lastBombsDropped;
    int dangerSteps = agentScript.lastSafetyDangerSteps;

    string line =
        $"{episode};{outcome};{deathType};{phase};{steps};{reward:F2};{epsilon:F4};
    " +

        $"{expRate:F2};{wInit};{wBroken};{bombs};{dangerSteps}";

    try
    {
        File.AppendAllText(csvFilePath, line + "\n");
    }
    catch (System.Exception e)
    {
        Debug.LogError("CSV Yazma Hatası: " + e.Message);
    }
}

```